САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа № 4

Развёртывание приложения
на удалённом сервере

Выполнил:

Корольков Артем

БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Подготовить удалённый сервер и установить необходимые зависимости. Развернуть на нём приложение из ЛР3 и настроить nginx как обратный прокси для доступа к приложению извне.

Ход работы

1 Подготовка ветки

Создал ветку lab4 на основе lab3 и отправил её в свой репозиторий GitHub. Для развёртывания использовал приложение из третьей лабораторной работы.

```
$ git switch --no-track -c lab4 origin/lab3 &&
git push -u origin lab4 &&
git status --short --branch
Switched to a new branch 'lab4'
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'lab4' on GitHub by visiting:
remote:      https://github.com/ArtemKorolkov/ITMO-ACS-Backend-2026-A/pull/new/l
ab4
remote:
To https://github.com/ArtemKorolkov/ITMO-ACS-Backend-2026-A.git
 * [new branch]      lab4 -> lab4
branch 'lab4' set up to track 'origin/lab4'.
## lab4...origin/lab4
```

Рисунок 1 — Создание и публикация ветки lab4

2 Проверка сервера

Подключился по SSH к серверу 31.76.16.48 под пользователем root. Проверил систему и ресурсы: установлена Ubuntu 24.04.4 LTS, доступно около 2 ГБ оперативной памяти и 20 ГБ диска.

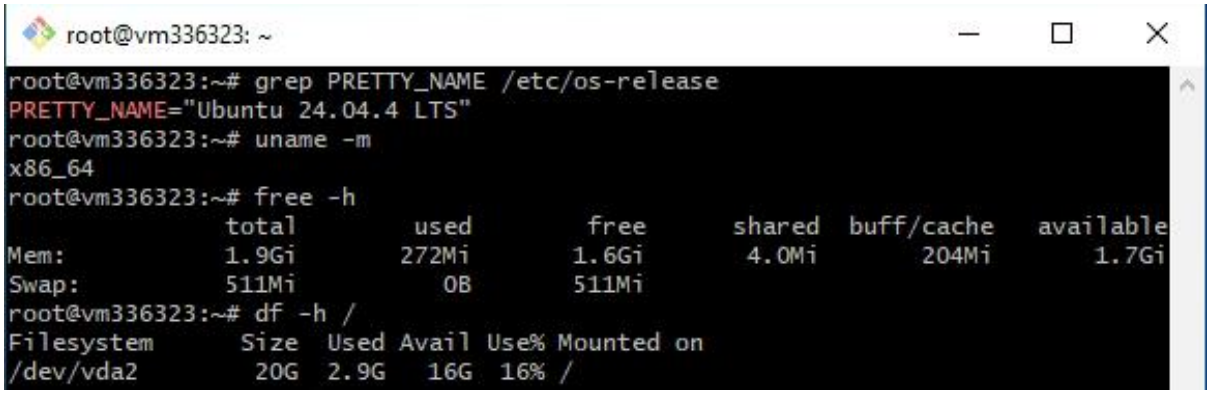


Рисунок 2 — Система и ресурсы удалённого сервера

3 Установка зависимостей

Обновил пакеты и установил Docker, плагины Compose и Buildx, nginx, Git и Python 3. Проверил версии программ и убедился, что службы Docker и nginx находятся в состоянии active.

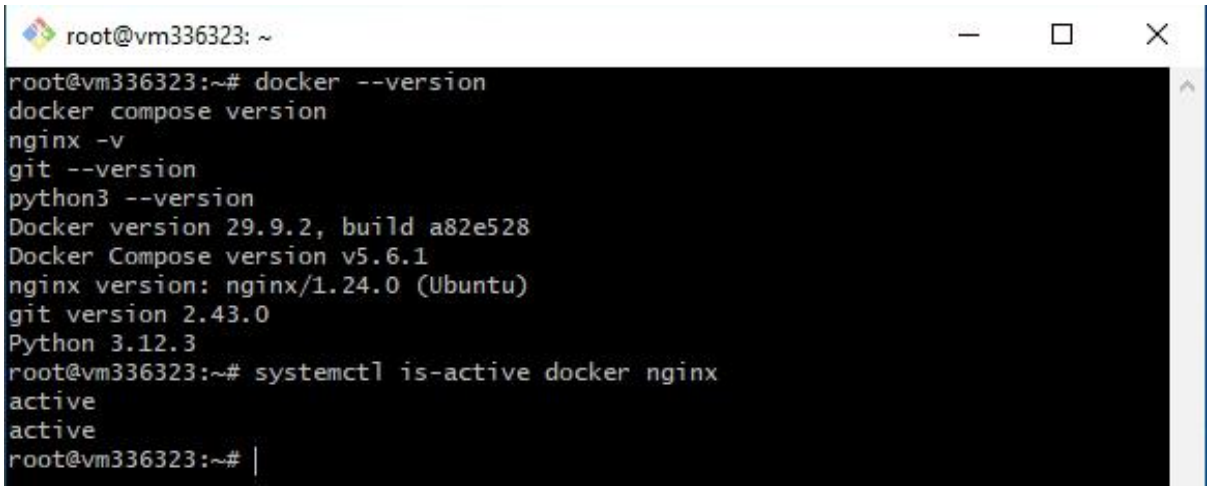


Рисунок 3 — Проверка зависимостей и состояния служб

4 Загрузка приложения

Склонировал ветку lab4 в каталог /opt/restaurant-lab4/repository на сервере. Проверил конфигурацию Docker Compose и получил список из RabbitMQ и четырёх сервисов приложения.

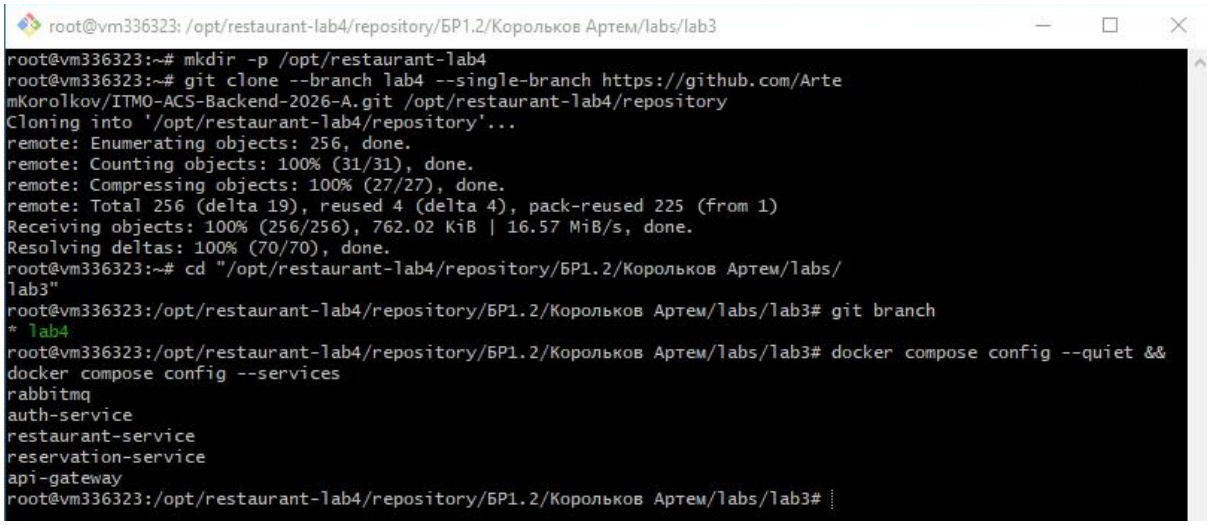


Рисунок 4 — Клонирование проекта и проверка Compose

5 Настройка переменных и данных

Создал отдельный файл app.env в каталоге /etc/restaurant-lab4 и сгенерировал значения JWT_SECRET, SERVICE_KEY и RABBITMQ_PASSWORD. Ограничил доступ к файлу правами 600, а для данных приложения создал каталог /var/lib/restaurant-lab4/data.

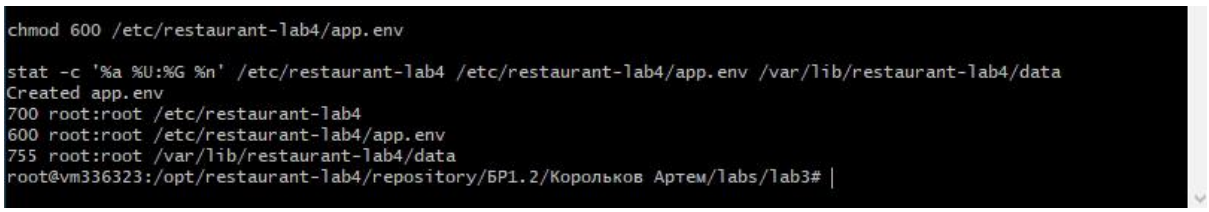


Рисунок 5 — Файл переменных окружения и права доступа

6 Конфигурация для сервера

Добавил файл compose.production.yml и скрипт compose.sh для запуска приложения на сервере. В настройках указал переменные окружения, хранение данных вне контейнеров, автоматический перезапуск, проверки состояния и ограничение размера журналов.

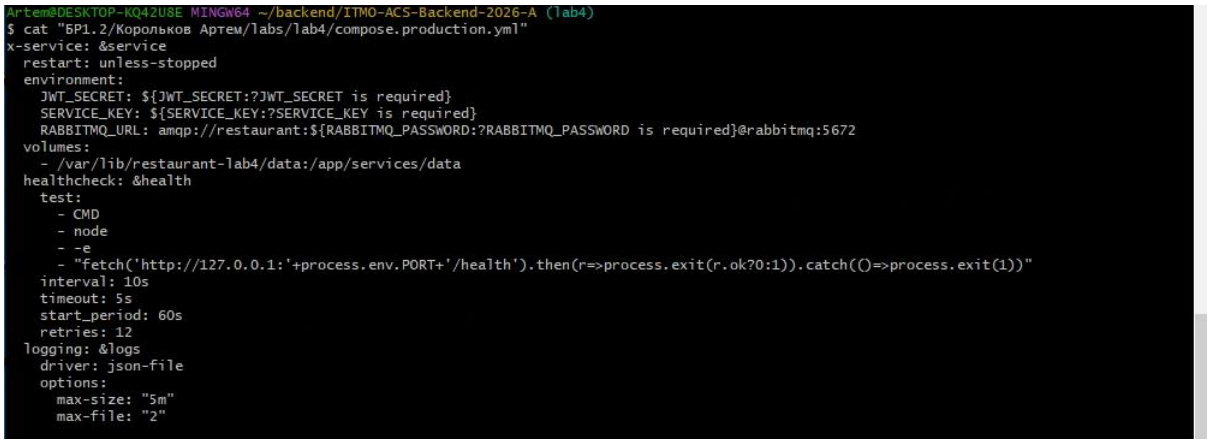


Рисунок 6 — Фрагмент конфигурации для развёртывания

7 Сборка приложения

Загрузил новые настройки на сервер и по очереди собрал образы auth-service, restaurant-service, reservation-service и api-gateway. Для сборки использовал Dockerfile каждого сервиса и команды из compose.sh.

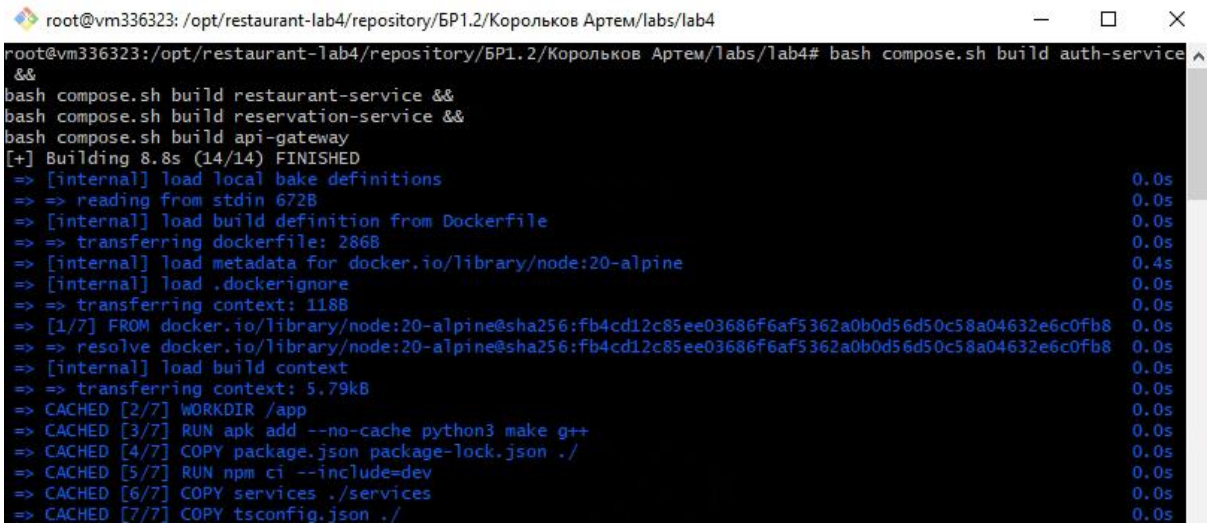


Рисунок 7 — Сборка образов сервисов

8 Запуск контейнеров

Запустил контейнеры и дождался завершения проверок состояния. Все пять контейнеров получили статус healthy, а порт API-шлюза 3010 доступен на сервере через адрес 127.0.0.1.

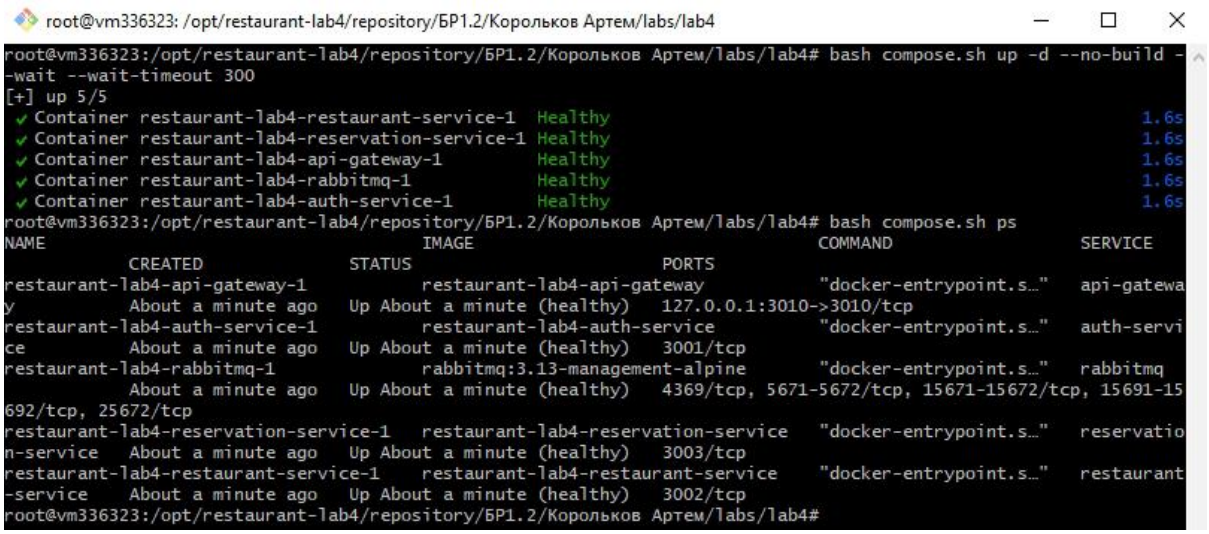


Рисунок 8 — Работающие контейнеры приложения

9 Настройка nginx

Настроил nginx на порту 80 как обратный прокси к http://127.0.0.1:3010. Проверил конфигурацию командой nginx -t: ошибок нет.

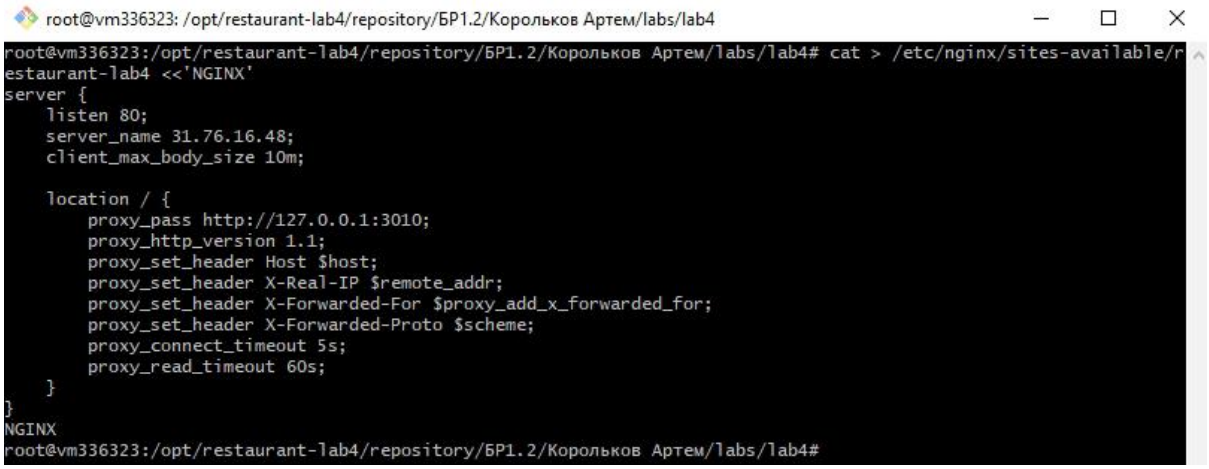


Рисунок 9 — Конфигурация nginx как обратного прокси

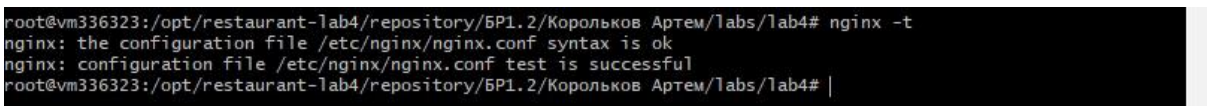


Рисунок 10 — Успешная проверка конфигурации nginx

10 Проверка приложения

Открыл http://31.76.16.48 в браузере и получил ответ приложения со статусом ok. Через nginx запрос /restaurants вернул HTTP 200, а запрос /users/me без токена вернул ожидаемый HTTP 401.

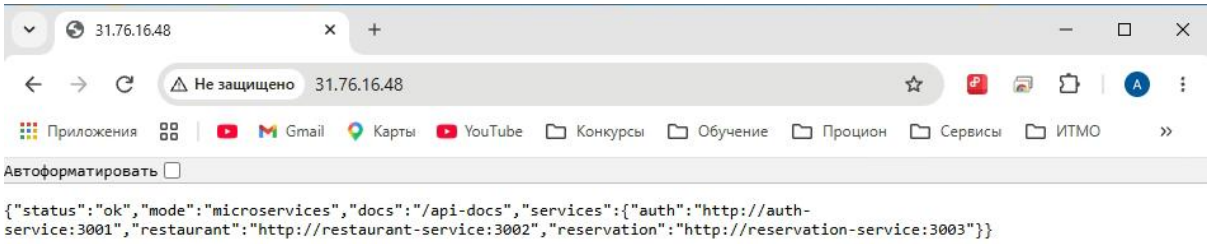


Рисунок 11 — Доступ к приложению по внешнему IP-адресу

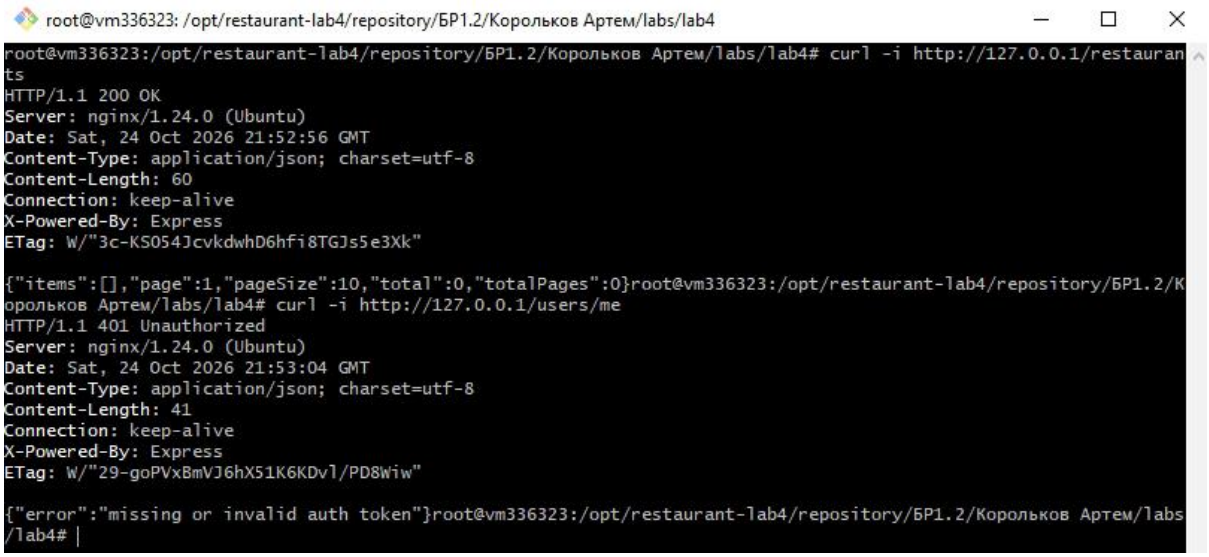


Рисунок 12 — Проверка открытого и защищённого маршрутов

Вывод

Подготовил удалённый сервер и развернул на нём приложение в Docker-контейнерах. Настроил nginx как обратный прокси и проверил доступ к приложению по внешнему IP-адресу.